LABORATORY OBSERVATION

Course: Full Stack Web Development
Course Code: 23CM4121
Experiment No.: 4
Student Name: P.Raghu Ram
Roll No.: A24126552102
Date: 30-08-2026

---

1. TITLE

Design and Implementation of Dynamic To-Do List Application Using JavaScript DOM Manipulation

2. AIM

To develop an interactive To-Do List web application using HTML5, CSS3, and JavaScript DOM manipulation to dynamically add, mark completed, delete tasks, and manage empty list state messages.

3. OBJECTIVE

The objectives of this experiment are:

- To design a styled task management container with input controls and action buttons.
- To register click event listeners (addEventListener('click', ...)) for adding new task items.
- To dynamically create HTML list elements (<li>, , <button>) using document.createElement.
- To implement interactive event handlers for toggling task completion (strikethrough styling) and task deletion (li.remove()).
- To maintain empty state messaging based on task list length (checkEmptyList()).

4. THEORY

Dynamic DOM manipulation enables real-time user interface updates without page reloads. Using JavaScript DOM methods such as document.createElement(), appendChild(), and element.remove(), elements can be dynamically added to or removed from an ordered/unordered list.

Event-driven programming allows binding functions to DOM nodes using addEventListener('click', callback). State transitions such as marking a task complete (updating textDecoration to 'line-through') or removing a task item directly update the document tree structure.

Application Flow:

User Enters Task -> Click 'Add' Button -> Input Validation -> Dynamic <li> & Button Creation -> Event Handler Binding (Complete / Delete) -> Element Appended to <ul> -> Empty Message Updated

5. ALGORITHM / PROCEDURE

1. Create an HTML5 document structure with a centered .todo-container holding heading 'My Tasks', input field, 'Add' button, empty list message, and an empty <ul> task list.
2. Style the container, inputs, buttons, list items, and completed state class (.completed) using CSS3 flexbox and box-shadows.
3. Retrieve DOM references for taskInput, addTaskBtn, taskList, and emptyMessage.
4. Define a checkEmptyList function that checks taskList.children.length and toggles emptyMessage display between 'block' and 'none'.
5. Attach a click event listener to addTaskBtn.
6. Read taskInput.value and trigger an alert if the input is empty.
7. Create new DOM nodes: <li> element, element for task text, 'Complete' button, 'Delete' button, and a wrapper for buttons.
8. Apply green background to 'Complete' button and red background to 'Delete' button.
9. Attach click listener to 'Complete' button to apply strikethrough (textDecoration = 'line-through') and gray color to task span.
10. Attach click listener to 'Delete' button to call li.remove() and invoke checkEmptyList().

11. Append buttons to buttonGroup, append span and buttonGroup to <li>, append <li> to taskList, clear taskInput value, and update empty list message state.

6. PROGRAM

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>My Dynamic To-Do List</title>
    <style>
body {
    font-family: Arial, sans-serif;
    background-color: #f4f4f9;
    display: flex;
    justify-content: center;
    padding-top: 50px;
}

.todo-container {
    background: white;
    padding: 20px;
    border-radius: 8px;
    box-shadow: 0 4px 8px rgba(0,0,0,0.1);
    width: 300px;
}

.input-section {
    display: flex;
    gap: 10px;
    margin-bottom: 20px;
}

input {
    flex: 1;
    padding: 8px;
}

button {
    padding: 8px 12px;
    background: #007bff;
    color: white;
    border: none;
    cursor: pointer;
}

ul {
    list-style: none;
    padding: 0;
}

li {
    background: #eee;
    margin-bottom: 10px;
    padding: 10px;
    display: flex;
    justify-content: space-between;
    align-items: center;
}

.completed {
    text-decoration: line-through;
    color: gray;
}
    </style>
</head>
<body>

    <div class="todo-container">
        <h1>My Tasks</h1>

        <div class="input-section">
            <input type="text" id="taskInput" placeholder="Add a new task...">
            <button id="addTaskBtn">Add</button>
        </div>
        <p id="emptyMessage">Your task list is currently empty.</p>
```

```
        <ul id="taskList">
        </ul>
    </div>

   <script>
    const taskInput = document.getElementById('taskInput');
    const addTaskBtn = document.getElementById('addTaskBtn');
    const taskList = document.getElementById('taskList');

    addTaskBtn.addEventListener('click', () => {
        const tasktext = taskInput.value;

        if(tasktext === ""){
            alert('add the any task to start');
            return;
        }
            function checkEmptyList() {
            if (taskList.children.length === 0) {
                emptyMessage.style.display = "block"; // Show message
            } else {
                emptyMessage.style.display = "none"; // Hide message
            }
        }

        const li = document.createElement('li');
        const span = document.createElement('span');
        const completeBtn = document.createElement('button');
        const deleteBtn = document.createElement('button');

        span.textContent = tasktext;

        completeBtn.textContent = "Complete";
        completeBtn.style.background = "rgb(112, 224, 0)";
        completeBtn.style.marginRight = "10px"; // Added a little gap

        deleteBtn.textContent = "Delete";
        deleteBtn.style.background = "#dc3545";

        completeBtn.addEventListener('click', () => {
            span.style.textDecoration = "line-through";
            span.style.color = "gray";
        });

        deleteBtn.addEventListener('click', () => {
            li.remove();
            checkEmptyList();
        });

        const buttonGroup = document.createElement('div');
        buttonGroup.appendChild(completeBtn);
        buttonGroup.appendChild(deleteBtn);

        li.appendChild(span);
        li.appendChild(buttonGroup);

        taskList.appendChild(li);

        taskInput.value = "";
        checkEmptyList();

    });
    checkEmptyList();
</script>
</body>
</html>
```

7. CODE EXPLANATION

Code / Syntax	Explanation
`document.getElementById('taskInput')`	Fetches input field reference to read entered task text string.
`addTaskBtn.addEventListener('click', ...)`	Attaches click event listener to handle task creation on button click.
`document.createElement('li')`	Dynamically creates list item node container for new task entry.
`completeBtn.addEventListener('click', ...)`	Toggles textDecoration property to 'line-through' for completed task styling.
`deleteBtn.addEventListener('click', ...)`	Removes list item element from DOM tree via li.remove().
`checkEmptyList()`	Toggles visibility of emptyMessage element based on taskList.children.length.
`taskList.appendChild(li)`	Appends newly constructed task element node as child of target element.

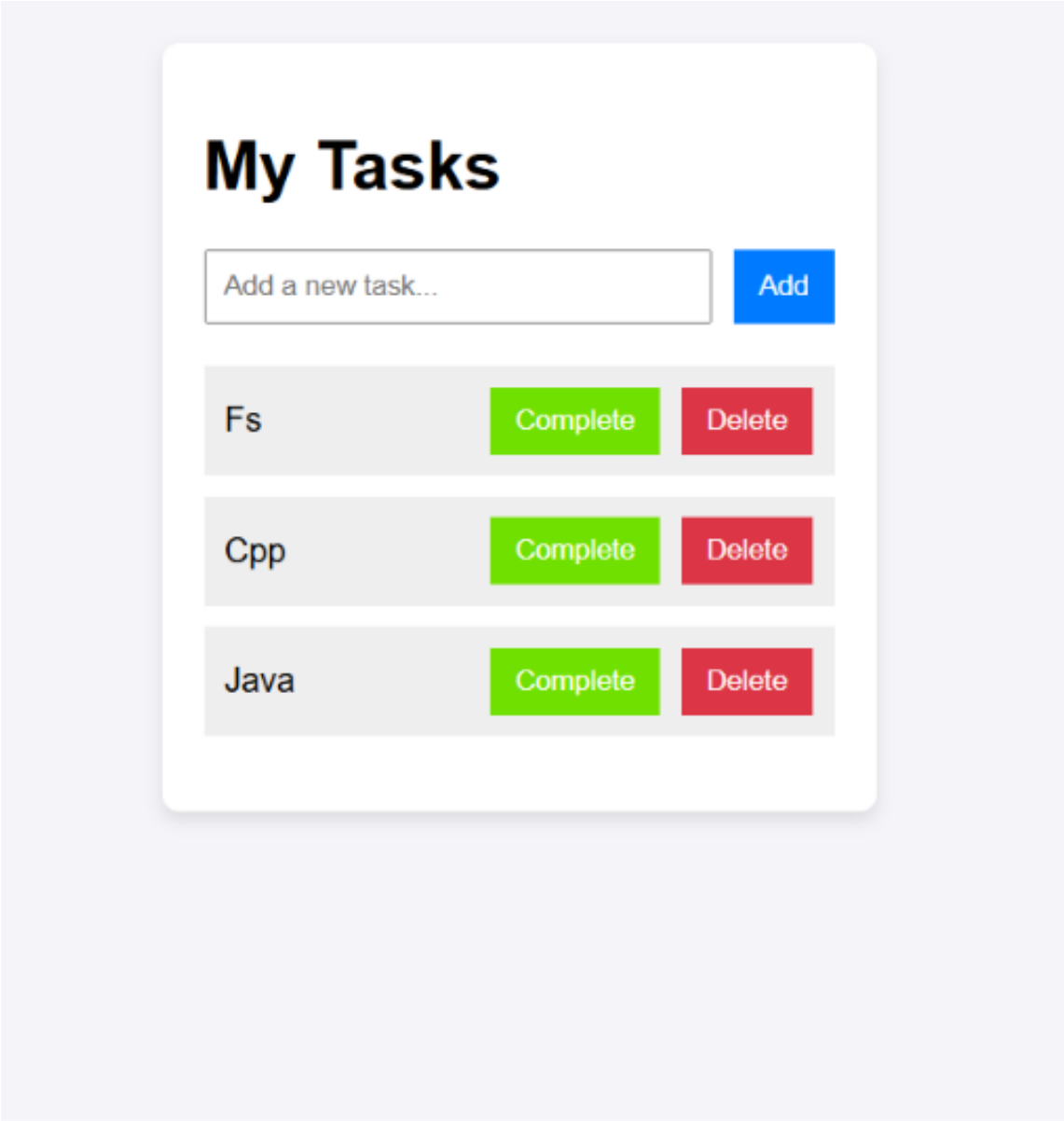
8. TEST CASES AND EXECUTION DATA

The experiment was executed with the following test cases to verify functionality:

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Initial Page Load	Open page in browser	'My Tasks' header, input box, and empty message display	**Pass**
TC-02	Empty Input Validation	Click 'Add' button without text	Alert 'add the any task to start' appears	**Pass**
TC-03	Add New Task Item	Enter task text & click 'Add'	Task item ('Fs', 'Cpp', etc.) added with Complete & Delete buttons	**Pass**
TC-04	Mark Task Completed	Click 'Complete' button	Task text styled with strikethrough line and gray color	**Pass**
TC-05	Delete Task Item	Click 'Delete' button	Task item removed from list; empty message shown if empty	**Pass**

9. OUTPUT

Output 1: Dynamic To-Do List Application Interface with Active Tasks



10. RESULT

Thus, the dynamic To-Do List application implementing interactive DOM element creation, completion toggling, deletion, and list state management was successfully designed, implemented, and verified.